

Code Review and Quality: Using AI to Improve Code Quality and Readability

NAME: P HARSHAVARDHAN

ROLL.NO:2503A523 66

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty

Python script.

Sample Input Code:

```
# Calculate average score of a student
def calc_average (marks):
```

Week5 -

Monday

```
total = 0
```

```
for m in marks:
```

```
total += m
```

```
average = total / len(marks)
```

```
return avrage # Typo here
```

```
marks = [85, 90, 78, 92]
```

```
print(&quot;Average Score is &quot;, calc_average (marks))
```

Expected Output:

Corrected and runnable Python code with explanations of the

fixes.

PROMPT:

Review this Python code, find syntax and logic errors, fix them, and give the corrected code with a short explanation.

INPUT CODE:

```
# Calculate average score of a student
```

```
def calc_average (marks):
```

```
    total = 0
```

```
    for m in marks:
```

```
        total += m
```

```
    average = total / len(marks)
```

```
    return average
```

```
marks = [85, 90, 78, 92]
```

```
print("Average Score is", calc_average (marks))
```

OUTPUT:

```
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> & "C:\Program Files\Python314\python.exe" "c:/Users/beebi/OneDrive/Desktop/ai assisted coding/lab10task1.py"
Average Score is 86.25
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> |
```

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect (L,B) : return L*B
```

```
print(area_of_rect(10,20))
```

Expected Output:

Well-formatted PEP 8-compliant Python code.

PROMPT:

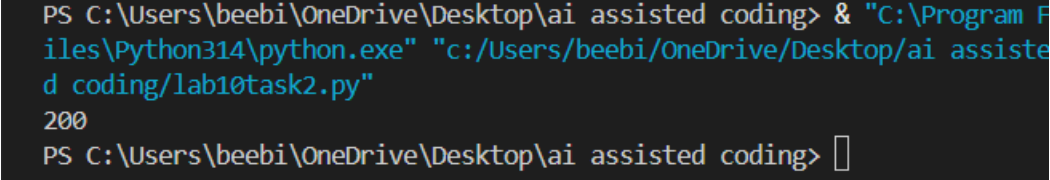
Refactor this Python code according to PEP 8 guidelines. Improve spacing, formatting, and readability without changing the logic.

INPUT CODE:

```
def area_of_rect(length, breadth):  
    return length * breadth
```

```
print(area_of_rect(10, 20))
```

OUTPUT:

A terminal window with a dark background. The command prompt is 'PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding>'. The command entered is '& "C:\Program Files\Python314\python.exe" "c:/Users/beebi/OneDrive/Desktop/ai assisted coding/lab10task2.py"'. The output shown is '200'. The prompt is followed by a cursor.

```
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> & "C:\Program F  
iles\Python314\python.exe" "c:/Users/beebi/OneDrive/Desktop/ai assiste  
d coding/lab10task2.py"  
200  
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> 
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its

logic.

Sample Input Code:

```
def c(x,y):  
    return x*y/100  
  
a=200  
b=15  
print(c(a,b))
```

Expected Output:

Python code with descriptive variable names, inline

comments, and clear formatting.

PROMPT:

Make this Python code more readable without changing its logic. Use descriptive variable names, clear formatting, and comments.

INPUT CODE:

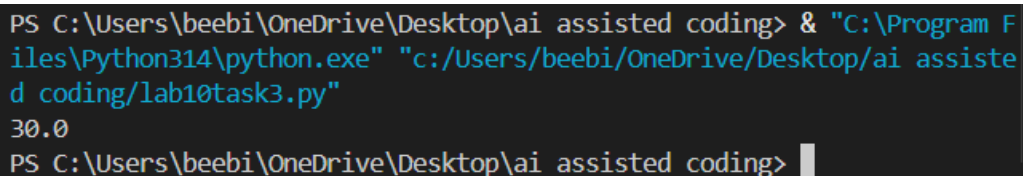
```
def calculate_percentage (value, percentage):  
    # Calculate the given percentage of a value  
    return value * percentage / 100
```

```
amount = 200
```

```
percentage = 15
```

```
print(calculate_percentage (amount, percentage))
```

OUTPUT:



```
PS C:\Users\beebe\OneDrive\Desktop\ai assisted coding> & "C:\Program Files\Python314\python.exe" "c:/Users/beebe/OneDrive/Desktop/ai assisted coding/lab10task3.py"  
30.0  
PS C:\Users\beebe\OneDrive\Desktop\ai assisted coding> █
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ['Alice', 'Bob', 'Charlie']
```

```
print('Welcome', students[0])
```

```
print('Welcome', students[1])
```

```
print('Welcome', students[2])
```

Expected Output:

Modular code with reusable functions.

PROMPT:

Refactor this Python code by removing repetition and creating a reusable function. Keep the same output.

INPUT CODE:

```
students = ["Alice", "Bob", "Charlie"]
```

```
def welcome_student (name):
```

```
    print("Welcome", name)
```

```
for student in students:
```

```
    welcome_student (student)
```

OUTPUT:

```
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> & "C:\Program Files\Python314\python.exe" "c:/Users/beebi/OneDrive/Desktop/ai assisted coding/lab10task4.py"
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> |
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

```
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
```

```
for n in nums:
    squares.append(n**2)
print(len(squares))
```

Expected Output:

Optimized code using list comprehensions or vectorized operations.

PROMPT:

Optimize this Python code for better performance and readability. Use an efficient list comprehension while keeping the same result .

INPUT CODE:

```
# Find squares of numbers
nums = range(1, 1000000)

squares = [n ** 2 for n in  nums]

print(len(squares))
```

OUTPUT:

```
PS C:\Users\beebe\OneDrive\Desktop\ai assisted coding> & "C:\Program Files\Python314\python.exe" "c:/Users/beebe/OneDrive/Desktop/ai assisted coding/lab10task5.py"
999999
PS C:\Users\beebe\OneDrive\Desktop\ai assisted coding>
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
    if score & gt;= 90:
        return & quot;A & quot; ;
    else:
        if score & gt;= 80:
            return & quot;B & quot; ;
        else:
            if score & gt;= 70:
                return & quot;C & quot; ;
            else:
                if score & gt;= 60:
                    return & quot;D & quot; ;
                else:
                    return & quot;F & quot; ;
```

Expected Output:

Cleaner logic using elif or dictionary mapping.

PROMPT:

Simplify this complex nested Python logic using elif statements. Keep the same grading logic and output.

INPUT CODE:

```
def grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    elif score >= 60:  
        return "D"  
    else:  
        return "F"  
print(grade(85))
```

OUTPUT:

```
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> & "C:\Program Files\Python314\python.exe" "c:/Users/beebi/OneDrive/Desktop/ai assisted coding/lab10task6.py"  
B  
PS C:\Users\beebi\OneDrive\Desktop\ai assisted coding> |
```